

SISTEMA DE PAQUETERIA

Proyecto de Programacion I - Java

Manual de usuario y documentacion tecnica

Dato	Informacion
Asignatura	Programacion I
Temas aplicados	Interfaces, excepciones y colecciones
Lenguaje	Java 17+
Interfaz	Java Swing
Persistencia	Memoria durante la ejecucion + exportacion CSV
Integrante principal	David Santiago Aguas Vanegas
Otros integrantes	Completar en caso de trabajo grupal

Documento preparado para la entrega academica. Incluye funcionamiento de la interfaz, estructura interna y forma de consumir la capa de servicio.

1. Descripcion general

El Sistema de Paqueteria permite registrar envios, consultar su estado, actualizar el seguimiento, despachar paquetes en orden de llegada, filtrar registros y exportar la informacion a un archivo CSV. El programa fue diseñado para practicar los conceptos de Programacion I trabajados en clase.

La arquitectura separa la interfaz grafica de la logica del negocio. La ventana Swing consume la clase **SistemaPaqueteriaService**; esta clase aplica validaciones y administra las colecciones que contienen los envios.

Nota de alcance: el proyecto no depende de una base de datos. Los datos viven en memoria mientras el programa esta abierto y se pueden exportar a CSV para conservar un reporte externo.

2. Funcionalidades

- **Registrar envio:** permite elegir modalidad Estandar o Express y capturar remitente, destinatario, ciudades, peso y valor declarado.
- **Generar codigo:** cada envio recibe un codigo de rastreo con formato PK-####.
- **Rastrear:** consulta un envio por codigo y presenta el historial de estados.
- **Actualizar estado:** controla las transiciones del ciclo de vida del envio.
- **Despachar:** utiliza una cola FIFO para atender el siguiente envio pendiente.
- **Listado:** permite ver todos los registros o filtrar por estado.
- **CSV:** exporta codigo, tipo, participantes, ruta, peso, valor, estado, costo y fecha.
- **Datos de demostracion:** carga tres envios para facilitar pruebas y sustentacion.

3. Estructura del proyecto

Paquete	Responsabilidad
model	Entidades, clase abstracta, interfaces y enum de estados.
service	Reglas del negocio y administracion de las colecciones.
exception	Excepciones propias para errores del dominio.
ui	Interfaz grafica en Java Swing.
util	Generacion de codigos y validaciones.
app	Punto de entrada y prueba de humo.

4. Aplicacion de los conceptos vistos en clase

El material de clase presenta la clase abstracta como una forma de compartir datos y comportamiento entre subclases y la interfaz como un contrato de metodos que pueden cumplir clases sin parentesco. El proyecto usa ambas ideas.

4.1 Clase abstracta y polimorfismo

```
public abstract class Envio implements Exportable, Rastreadable {
    private final String codigo;
    ...
    public abstract double calcularCosto();
    public abstract String getTipo();
}
```

Las clases **PaqueteEstandar** y **PaqueteExpress** heredan de **Envio** y calculan el costo de forma diferente. Esto permite trabajar con referencias del tipo **Envio** sin conocer la clase concreta.

4.2 Interfaces

```
public interface Exportable {
    String aCSV();
}

public interface Rastreadable {
    void actualizarEstado(EstadoEnvio nuevoEstado);
    String generarReporteRastreo();
}
```

La interfaz **Exportable** define el contrato para producir una fila CSV y **Rastreadable** define las operaciones de seguimiento. El contrato se implementa en la clase **Envio** y se hereda a sus especializaciones.

4.3 Excepciones

```
public class DatoInvalidoException extends PaqueteriaException {
    public DatoInvalidoException(String message) {
        super(message);
    }
}
```

Se utilizan excepciones propias para avisar problemas del dominio, como datos vacios, codigo de envio no encontrado y estados invalidos. La interfaz captura la excepcion y presenta el mensaje al usuario.

4.4 Colecciones

```
private final Map<String, Envio> enviosPorCodigo = new LinkedHashMap<>();
private final List<Envio> historialEnvios = new ArrayList<>();
private final Set<String> ciudadesDestino = new HashSet<>();
private final Queue<String> colaDespacho = new ArrayDeque<>();
```

La eleccion sigue la operacion requerida: **Map** para localizar por codigo, **List** para conservar el historial, **Set** para evitar ciudades repetidas y **Queue** para atender por orden de llegada.

5. Interfaz grafica

La pantalla principal esta dividida en cuatro pestañas: Registrar envio, Rastrear, Operaciones y Listado y CSV.

SISTEMA DE PAQUETERIA

Registrar envío

Rastrear

Operaciones

Listado y CSV

Remitente:

Destinatario:

Ciudad origen:

Ciudad destino:

Peso (kg):

1

Valor declarado (COP):

50,000

Tipo de envío:

Estandar

Registrar envío

Cargar datos de ejemplo

Limpiar

Figura 1. Ventana principal del sistema.

6. Manual de uso

6.1 Registrar un envío

- Ingrese remitente y destinatario.
- Digite ciudad de origen y ciudad de destino.
- Indique el peso y valor declarado.
- Seleccione Estandar o Express.
- Pulse **Registrar envío**. El sistema muestra el código de rastreo y el costo calculado.

6.2 Rastrear un envío

- Abra la pestaña **Rastrear**.
- Escriba el código, por ejemplo PK-1000.
- Pulse **Consultar**.
- La caja de texto presenta tipo, ruta, estado actual e historial.

6.3 Actualizar estado

Las transiciones implementadas son: REGISTRADO -> EN_TRANSITO -> EN_CENTRO_DISTRIBUCION -> EN_REPARTO -> ENTREGADO. En varios estados también se permite CANCELADO. Las transiciones no permitidas generan una excepción propia y se informa al usuario.

6.4 Despachar

El botón **Despachar siguiente** extrae de la cola el primer envío pendiente y lo mueve a EN_TRANSITO. La operación representa una cola FIFO.

6.5 Filtrar y exportar

En la pestaña **Listado y CSV** se puede filtrar por estado. El botón **Exportar CSV** crea el archivo **data/envios.csv**.

7. Como consumir el servicio

La aplicación sigue una separación sencilla entre interfaz y lógica. Un componente cliente, en este caso MainFrame, crea o recibe una instancia de SistemaPaqueteriaService y llama sus métodos públicos.

```
SistemaPaqueteriaService service = new SistemaPaqueteriaService();
Envio envio = service.registrarEnvio(
    false, "Ana", "Carlos", "Bogota", "Medellin", 2.5, 180000
);
service.actualizarEstado(envio.getCodigo(), EstadoEnvio.EN_TRANSITO);
Envio consulta = service.buscarPorCodigo(envio.getCodigo());
service.exportarCSV(Path.of("data", "envios.csv"));
```

Los métodos del servicio constituyen la API interna del proyecto. La interfaz gráfica no manipula directamente las colecciones; solicita operaciones al servicio y maneja las excepciones que este puede producir.

8. Reglas principales del dominio

Regla	Comportamiento
Datos obligatorios	Remitente, destinatario, origen y destino no pueden quedar vacíos.
Peso y valor	Deben ser mayores que cero.
Código	Se genera automáticamente y se utiliza como clave del Map.

Regla	Comportamiento
Estados	No se permite saltar arbitrariamente entre estados.
Envio cerrado	Un envio ENTREGADO o CANCELADO no vuelve a abrirse.
Despacho	Se respeta el orden de llegada de la cola.

9. Pruebas realizadas

Se realizo una prueba automatizada de humo que verifica: registro de un envio, consulta por codigo, cambio de estados, estado final ENTREGADO y exportacion de un archivo CSV temporal. El resultado fue satisfactorio.

SMOKE TEST OK: registro, rastreo, transiciones y exportacion CSV funcionando.

Ademas, el proyecto se compilo correctamente con JDK 21 en el entorno de construccion. El codigo se mantiene compatible con Java 17 en el pom.xml.

10. Ejecucion del proyecto

Opcion A - Windows: ejecutar **run.bat**. El script compila las clases y abre la interfaz.

Opcion B - Linux/macOS: ejecutar **./run.sh**.

Opcion C - Maven: usar **mvn clean compile** y luego **mvn exec:java**.

11. Git y rama entregable

La carpeta de entrega esta preparada como repositorio Git y utiliza una rama llamada **entregable**. Para conectarla a un repositorio remoto, agregue la URL de GitHub/GitLab y ejecute:

```
git remote add origin URL_DEL_REPOSITORIO
git push -u origin entregable
```

La regla indicada en el taller es mantener la rama entregable sin commits posteriores a la fecha de entrega. Por eso, despues de publicar la entrega se debe evitar realizar nuevos commits sobre esa rama.

12. Relacion con el material de clase

La solucion adopta directamente los cuatro ejes destacados en el material: clase abstracta para una familia de objetos relacionados, interfaces para contratos de comportamiento, excepciones para responder ante fallos y colecciones elegidas segun la operacion que el sistema repite.

13. Archivos de entrega

Ruta	Contenido
src/main/java/...	Codigo fuente organizado por paquetes.
pom.xml	Configuracion Maven y version de Java.
run.bat / run.sh	Scripts de ejecucion rapida.
README.md	Instrucciones de instalacion, ejecucion y Git.
docs/Manual_Usuario_y_Tecnico.pdf	Esta documentacion.
data/	Ubicacion donde la aplicacion genera envios.csv.

Fin del documento.